

Django - 1

Main outcome

A browser sends an HTTP request to our server. Django receives that request, finds the correct view using the URL configuration, calls our Python view function, and sends the returned HTTP response back to the browser.

```
http://127.0.0.1:8000/  
http://127.0.0.1:8000/about/
```

Until now, how have our Python programs received input?

Students might say:

```
name = input("Name: ")  
print(f"Hello {name}")
```

```
USER  
↓  
terminal  
↓  
Python program  
↓  
terminal  
↓  
USER
```

A web application follows the same general principle.

Something sends input.

Our program processes it.

Our program produces output.

But instead of:

```
terminal → Python → terminal
```

we have:

```
browser → network → Python web application → network → browser
```

This is the bridge from Python programming into backend development.

What exactly is a website?

HTML → content/structure
CSS → appearance
JavaScript → behavior inside the browser

But imagine a blog.

Someone visits:

`/posts/42/`

How does the browser know what Post 42 contains?

It doesn't.

That information might exist in a database on another computer.

Something must:

```
receive "give me post 42"  
  ↓  
find post 42  
  ↓  
generate a page  
  ↓  
send it back
```

That is one major responsibility of the **backend**.

Frontend vs Backend

Frontend

Code/content operating primarily on the user's side.

For example:

HTML
CSS
browser-side JavaScript

Backend

Software running on the server that handles things such as:

```
requests
business rules
database access
authentication
authorization
generating responses
```

For a blog:

```
Browser                                Django
"Show me posts" ----->
                                Find posts
                                Generate response
HTML page <-----
```

Client and Server

Client

A program asking another program for something.

Today our client is usually:

```
Chrome
Firefox
Edge
Safari
```

Server

A program that listens for requests and sends responses.

Important clarification:

"Server" can refer to a computer, but in web programming it can also refer to software listening for network requests.

So:

```
browser = client
Django application = server-side application
computer = running both
```

HTTP without overcomplicating it

The browser and server need agreed rules for communicating.

One important protocol used for web communication is:

HTTP — Hypertext Transfer Protocol.

REQUEST
RESPONSE
METHOD
PATH
STATUS CODE
BODY

Example request conceptually:

GET /about/

Meaning:

Please give me the resource at /about/.

A response might conceptually contain:

Status: 200 OK

<h1>About us</h1>

GET

The browser is requesting information.

/about/

Which resource it wants.

200

The request succeeded.

Response body

The actual content returned.

What happens after typing a URL?

1. User enters URL
 - ↓
 2. Browser creates HTTP request
 - ↓
 3. Request reaches server
 - ↓
 4. Django examines URL
 - ↓
 5. Django finds a view
 - ↓
 6. View executes Python
 - ↓
 7. View returns response
 - ↓
 8. Django sends response
 - ↓
 9. Browser displays it
-

What is Django?

Without a framework, we'd need to solve many common web-development problems ourselves:

How do we route URLs?
How do we read requests?
How do we return responses?
How do we communicate with databases?
How do we validate forms?
How do users log in?
How do sessions work?
How do we protect against common web attacks?

A **web framework** provides reusable structures and tools for solving these recurring problems.

Django is a Python web framework.

So we'll still write Python:

```
def home(request):  
    ...
```

but Django provides the web infrastructure surrounding our Python.

Django's architecture

Just show the map.

```
Request
  ↓
URL
  ↓
View
  ↓
Model
  ↓
Database

View
  ↓
Template
  ↓
Response
```

First:

URL + View

Later:

Template

End:

Model + Database

Why do we need a package manager?

Previously done:

```
import random
```

That's standard-library code.

But Django is an external package.

Our project therefore has dependencies.

Example:

```
Our project
├── requires Django
```

If another student downloads our project, they need to know:

```
which Python?
which Django?
which dependencies?
which versions?
```

That's where project/dependency tooling helps.

What is uv?

uv will manage the Python environment and external packages for this project.

The current uv project workflow stores project requirements in `pyproject.toml`; it creates an isolated `.venv` project environment and maintains exact resolved dependency information in `uv.lock`. `uv add` adds dependencies and `uv run` runs commands using that managed project environment.

```
Computer
├── Project A
│   ├── .venv
│   │   └── its packages
└── BeginnerBlog
    ├── .venv
    │   └── Django
```

One project might need:

```
Django 6.1
```

Another project could need a different version.

We don't want package choices from one project unexpectedly affecting another.

Create the project

1. Create our folder/project metadata

```
uv init --bare beginner-blog
cd beginner-blog
```

--bare is useful here because it asks uv to create only the `pyproject.toml`, avoiding an unrelated starter Python `src/` structure before Django generates its own project structure.

Standardize Python (Optional)

```
uv python pin 3.12
```

Explain:

We're telling the project which Python version we expect.

They'll see:

```
.python-version
```

Later their folder will conceptually look like:

```
beginner-blog/
├── .python-version
└── pyproject.toml
```

Install Django

```
uv add "Django==6.1"
```

```
pyproject.toml
uv.lock
.venv/
```

pyproject.toml

What our project says it depends on.

`uv.lock`

The exact dependency resolution uv selected.

`.venv`

The isolated Python environment containing the packages.

This matches uv's documented project model.

4. Verify Django

```
uv run python -m django --version
```

Installed Django version

Why do we write `uv run`?

Because we want the command to run using this project's managed environment and dependencies.

uv verifies the project's lockfile and environment before `uv run` executes the requested command.

Create a Django project

Run:

```
uv run django-admin startproject config .
```

```
uv run
```

Run inside our uv-managed project environment.

```
django-admin
```

Django's command-line utility.

```
startproject
```

Generate a Django project.

config

Name of the Django configuration package.

.

Create it in our current directory.

The official Django starter generates `manage.py` plus a project package containing settings, URL configuration, ASGI and WSGI entry points.

Now:

```
beginner-blog/
├── .python-version
├── .venv/
├── config/
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├── manage.py
├── pyproject.toml
└── uv.lock
```

Project files

`manage.py`

Remote control for our Django project.

We'll use it for commands such as:

```
uv run python manage.py runserver
uv run python manage.py startapp
uv run python manage.py migrate
uv run python manage.py createsuperuser
```

`settings.py`

Configuration.

Eventually contains information about:

```
installed apps
database
```

templates
middleware
static files
security settings

urls.py

The site's URL map.

Example idea:

/	→ homepage
/about/	→ about page
/posts/	→ posts
/admin/	→ administration

asgi.py and wsgi.py

Tell beginners:

These are interfaces used by servers to communicate with Django. We do not need to understand their internals yet.

Run the server

```
uv run python manage.py runserver
```

Then visit:

`http://127.0.0.1:8000/`

What just happened?

Django started a development web server.

↓

It listens for requests.

↓

Our browser sent it a request.

↓

Django returned a response.

Django documents `runserver` as a lightweight **development** server and explicitly warns not to use it as the production server.

127.0.0.1

means, in this context:

This computer itself.

localhost
8000

is the port Django's development server is currently listening on.

Analogy:

IP address = building
port = particular door/service

Create our Blog application

A **Django project** is the overall website configuration.

A **Django app** is a component responsible for some functionality.

For us:

```
PROJECT
BeginnerBlog

    contains

APP
blog
```

A larger project might look like:

```
Project
├── blog
├── accounts
├── shop
└── payments
```

Django's own documentation makes this distinction: a project can contain multiple apps, while an app represents some unit of web functionality.

Run:

```
uv run python manage.py startapp blog
```

Now:

```
blog/
├── __init__.py
├── admin.py
├── apps.py
├── migrations/
├── models.py
├── tests.py
└── views.py
```

Register the app

Open:

```
config/settings.py
```

Find:

```
INSTALLED_APPS = [
```

Add:

```
"blog",
```

Creating a Python folder does not automatically mean the Django project should treat it as an installed Django application. `INSTALLED_APPS` tells Django which applications participate in the project.

Write the first view

Open:

```
blog/views.py
```

Write:

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("<h1>Welcome to BeginnerBlog</h1>")
```

Now connect this directly to Python.

You already understand:

```
def greet(name):  
    return f"Hello {name}"
```

A Django view is also a Python callable.

Conceptually:

```
def home(request):  
    return response
```

The important difference is what Django gives it and what Django expects back.

request

An object containing information about the incoming HTTP request.

HttpResponse

An object representing what we're sending back.

Django's introductory tutorial uses exactly this basic pattern: a view receives `request` and returns an `HttpResponse`.

But how does `/` find `home()` ?

Right now we have:

```
def home(request):  
    ...
```

But Django doesn't magically know which URL should call it.

We need:

URL → view

Create:

blog/urls.py

with:

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [  
    path("", views.home, name="home"),  
]
```

The empty path

""

Inside this app, the empty path.

views.home

The Python function Django should call.

name="home"

A name we'll later use to refer to this URL without hard-coding it everywhere.

Connect the app URLs to the project

Open:

config/urls.py

Use:

```
from django.contrib import admin  
from django.urls import include, path
```

```
urlpatterns = [  
    path("", include("blog.urls")),  
    path("admin/", admin.site.urls),  
]
```

Explanation of include()

```
config/urls.py  
|  
| says:  
| "blog handles these URLs"  
↓  
blog/urls.py  
|
```

```
    | says:
    | "" uses home()
    ↓
blog/views.py
```

Django's URL dispatcher uses exactly this project-URLconf → included app-URLconf → view pattern; `include()` delegates the remaining part of the path to another URL configuration.

Trace the complete request

Now visit:

`http://127.0.0.1:8000/`

On the board:

```
Browser
  |
  | GET /
  ↓
Django development server
  ↓
config/urls.py
  |
  | ""
  ↓
blog/urls.py
  |
  | ""
  ↓
views.home(request)
  ↓
HttpResponse(...)
  ↓
Django
  ↓
Browser
```

Build `/about/`

Now let them follow you once.

In:

`blog/views.py`

add:

```
def about(request):  
    return HttpResponse(  
        "<h1>About BeginnerBlog</h1>"  
        "<p>We are learning Django.</p>"  
    )
```

Then:

```
urlpatterns = [  
    path("", views.home, name="home"),  
    path("about/", views.about, name="about"),  
]
```

Visit:

`http://127.0.0.1:8000/about/`

Trace it:

```
GET /about/  
  ↓  
config URL  
  ↓  
blog URL  
  ↓  
about()  
  ↓  
HttpResponse
```

Deliberately create a 404

Visit:

`http://127.0.0.1:8000/contact/`

They should see:

404

Ask:

Is Django broken?

No.

The server received the request correctly.

The problem is:

`/contact/`

doesn't match a URL pattern.

This is extremely important.

A 404 is a **response**, not evidence that the whole backend died.

The development terminal

Point out requests appearing in the terminal.

For example conceptually:

```
GET / HTTP/1.1 200
GET /about/ HTTP/1.1 200
GET /contact/ HTTP/1.1 404
```

Now connect:

GET	HTTP method
/about/	path
200	successful response
404	resource wasn't found

Day 1 exercise

Exercise: Add a contact page

Requirements:

Create:

`/contact/`

It should display:

```
<h1>Contact Us</h1>
```

<p>Email: hello@example.com</p>

They must decide which files need changing.

Expected realization:

```
blog/views.py
blog/urls.py
```

Not:

```
settings.py
config/urls.py
```

because the app is already connected.

Second exercise

Create:

```
/hello/
```

returning:

```
Hello from Django!
```

Then ask them to explain aloud:

What happens from the moment `/hello/` enters the browser until the text appears?

That verbal explanation is more valuable than another 20 lines of code.

Debugging exercise

Given this:

```
urlpatterns = [
    path("about/", views.about),
]
```

but:

```
def about_page(request):
    return HttpResponse("About")
```

What is wrong?

```
views.about
```

doesn't exist.

Then fix:

```
path("about/", views.about_page)
```

This reinforces that URL routing ultimately points at ordinary Python functions.

Vocabulary sheet

They should know these definitions before leaving.

Term	Beginner definition
Frontend	The part of the web application primarily presented/executed in the user's browser
Backend	Server-side code handling requests, application logic, data and responses
Client	A program making a request
Server	A program accepting requests and producing responses
HTTP	Rules used for communication between web clients and servers
Request	A message asking the server to perform something
Response	The server's answer
GET	HTTP method commonly used to request information
URL	Address identifying a web resource
Path	The URL portion such as <code>/about/</code>
Port	Identifies a particular network service on a machine
Framework	Reusable structure/tools for building applications
Django	A Python web framework
Project	Overall Django website configuration
App	A Django component responsible for a particular area of functionality
View	Python callable handling a request and producing a response
URLconf	Django configuration mapping URL patterns toward views
Dependency	External software/package our application requires
Virtual environment	Isolated Python environment for one project

Concept check

1.

Does the browser execute our Django Python view?

No.

Django runs on the server side.

2.

What does the browser send Django?

An HTTP **request**.

3.

What should a Django view produce?

An HTTP **response**.

4.

What connects `/about/` to our Python function?

The URL configuration.

5.

Is `home()` still a Python function?

Yes.

6.

What is the difference between a Django project and app?

Project:

Overall website/configuration.

App:

A component providing some particular functionality.

7.

Why are we using uv?

To manage the project's Python environment, dependencies and reproducible dependency versions.

8.

What is `.venv`?

An isolated Python environment belonging to this project.

9.

What does a 404 mean?

The requested resource/path could not be found by the application.

10.

What does `200` generally indicate?

The request succeeded.

Task 1

Add:

```
/services/  
/team/  
/faq/
```

Each needs its own view.

Task 2

Make / display:

```
Welcome to My Blog  
Learning Django one step at a time.
```

Task 3

Draw this architecture **from memory**:

```
Browser  
  ↓ request  
URL configuration  
  ↓  
View  
  ↓  
Response  
  ↓  
Browser
```

```
beginner-blog/  
├── blog/  
│   └── migrations/
```

```
config/urls.py:
```

```
from django.contrib import admin
from django.urls import include, path
```

```
urlpatterns = [
    path("", include("blog.urls")),
    path("admin/", admin.site.urls),
]
```